

沈进坤面试题

岗位目标：系统架构师 / 技术总监 | 面试官参考文档

候选人预评

基于简历分析，以下为面试前对候选人各维度的预期评估，供面试官参考——实际评分应在面试后填写。

维度	权重	简历匹配点	预期强项	潜在风险点	面试后评分（10分）
架构能力	25%	Eureka→Nacos 迁移、SOA 改造、微服务拆分	有完整架构演进经验，横跨 Java/.NET 双生态	简历未展现大规模高并发架构（如千万 QPS）经验	4-5
技术深度	20%	JVM 调优、多语言全栈、区块链底层	技术栈广度足，17 年沉淀	深度是否匹配总监级期待验证（如 JIT、GC 底层）	4-5
数据能力	20%	SQL Server/MySQL/MongoDB/ES 多存储实践	跨数据库经验丰富，有 Redis 缓存层设计经验	大数据量下的分库分表实战深度待确认	4
工程能力	20%	CI/CD、Docker、Spring Cloud 全家桶	DevOps 全流程经验，有容器化落地实践	是否有 K8s 生产运维经验简历未明确	/
领导力	15%	管理过 50 人全功能团队，KPI/KPA/OKR 实践	管理经验扎实，有过多种考核模式切换	跨部门协调、向上管理的复杂度待验证	/

面试结论

总体评价：做过，但仅限做过——17 年工龄未体现出与之匹配的技术深度。

面试实际只覆盖了前三项核心维度（架构能力、技术深度、数据能力），之后面试官已判定无需继续考察工程能力和领导力。各维度具体表现如下：

架构能力（4-5 / 10）：候选人对 Eureka、Nacos、Spring Cloud 等中间件有使用经验，也确实参与过架构迁移项目。但当追问到 CAP 模型在不同注册中心的工程语义差异、双向同步的一致性边界、以及在高并发/大数据量下的架构取舍时，回答停留在"操作层面"——知道怎么配、怎么迁，但讲不清楚为什么这样选、换一种会怎样。认知深度与"架构师/技术总监"的岗位预期有显著差距。

技术深度（4-5 / 10）：JVM 相关追问（GC 调优、JIT 编译、逃逸分析）暴露出知识碎片化的问题。候选人对 GC 的基本概念有了解，但无法从 GC 日志反推根因、给不出明确的调优决策链路；JIT 和 deoptimization 方向基本无实战经验。综合判断：技术知识更像"项目用到什么学什么"，缺乏系统性的底层原理框架，17 年工龄下这尤为突出。

数据能力（4 / 10）：候选人使用过多种数据库（SQL Server、MySQL、MongoDB、ES），但深入追问时发现——对 MySQL 优化器的行为细节（如 index dive、统计信息采样）、SQL Server 与 MySQL 的优化器差异、以及大数据量下的分库分表工程方案缺乏系统性认知。数据库认知停留在"能写 SQL、能建索引"的层次，与 17 年工龄预期的数据库选型和调优深度不吻合。

工程能力（/）：前三项面完后已判定不符合岗位要求，未继续考察。

领导力（/）：前三项面完后已判定不符合岗位要求，未继续考察。

核心问题归纳：该候选人的职业轨迹覆盖了广泛的技术栈和多个项目角色，但每次深入追问都停留在"我做过这个"而非"我理解这个"。对于架构师/技术总监岗位而言，"做过"只是起点，从"做过"中提炼出可迁移的判断原则、能从原理层解释现象、能在 trade-off 中给出有理有据的决策——这些才是岗位要求的核心能力。本轮面试未能验证候选人在上述维度达标。

结论：不通过。

面试重点关注方向（预备参考资料，本轮未全部启用）

1. **架构决策的 trade-off 思维：**能否在 CAP、性能 vs 一致性、微服务拆分 vs 运维复杂度之间做出有理有据的权衡，而非简单选边。
2. **技术深度的真实边界：**JVM 底层、分布式一致性、数据库优化器行为——这些是区分"会用框架"和"理解原理"的分水岭。

3. **商业思维**：架构师/总监不只是解决技术问题，更要把技术投入翻译成业务收益，能否说服非技术决策者是关键。
4. **坦诚度**：经历过大规模系统的人一定有失败案例，重点关注能否坦诚复盘教训而非只讲成功。

面试节奏建议

阶段	题目	时长	目的
暖场	简历项目深挖（10 分钟自由提问）	10 min	建立信任，观察表达能力和真诚度
核心技术	题 1 + 题 3 + 题 5（必答）	45 min	检验架构 + JVM + 数据库三大核心能力
工程与领导力	题 7 + 题 9（必答）	35 min	检验微服务落地经验和管理成熟度
补充深挖	根据前面表现选 2-3 题	20 min	针对性补刀薄弱维度
候选人提问	开放	10 min	观察候选人关注什么（技术/业务/团队/薪资）

第一部分：系统架构与分布式（权重 25%）

题 1：注册中心选型 — Eureka → Nacos 迁移

你在数字权益平台项目中做过 Eureka 到 Nacos 的迁移，用了 NacosSync 做过渡。请展开讲：

主问题：Eureka AP 模型和 Nacos 同时支持 AP/CP 模型，在实际迁移过程中，你们是如何处理服务发现的语义差异的？迁移期间，有没有遇到过"注册中心数据不一致导致流量打到已下线节点"的问题，怎么解决的？

追问方向：

- NacosSync 在双向同步时，如果两边同时有注册/注销操作，冲突如何仲裁？
- Eureka 自保护模式下，所有实例都"可用"——你当时怎么判断真实的下线节点？
- 如果让你重来一次，你会选 Nacos 的 AP 还是 CP 模式，为什么？

评分标准：

- 只答"NacosSync 自动同步"
 - 能说出 CAP 在注册中心的具体体现和双写冲突
 - 能结合生产故障案例说明自保护机制的坑和双向同步的一致性边界
-

题 2：分布式一致性实践

你在 108 社区和智慧城市 SaaS 平台中都用了 RabbitMQ/RocketMQ 做异步通讯。

主问题：假设有一个"用户下单后扣减库存并发放积分"的场景，订单服务、库存服务、积分服务都在独立的数据库中。你们如何保证这三个操作的数据最终一致性？具体用过什么模式，踩过什么坑？

追问方向：

- RocketMQ 的事务消息和 RabbitMQ 的 publisher confirm + 本地消息表，你分别会在什么场景选哪个？
- 如果消费方处理成功了，但发送方本地事务回滚了怎么办？
- 你们的对账机制是怎么做的？T+1 离线对账还是准实时补偿？

评分标准：

- 只答"用 MQ 异步"
 - 能对比不同方案的事务保证力度和适用场景
 - 能从"消息发送+本地事务非原子"出发，讲清半消息/事务回查/补偿的完整链路
-

第二部分：JVM 与性能调优（权重 20%）

题 3：GC 策略实战

主问题：假设你们 108 社区的线上服务出现了偶发性 Full GC，每次约 2-3 秒，导致部分请求超时。请完整描述你的排查和解决思路——从发现问题到最终修复的全过程。

追问方向：

- 你怎么确定是 GC 导致的 RT 抖动，而不是其他原因（如数据库慢查询、网络抖动）？
- 有没有遇到过"调了 GC 参数后吞吐量反而下降"的情况？为什么？
- 你实际项目中最常使用的 GC 组合是什么？如果给你一个 8G 堆、对延迟敏感的服务，你会怎么配参数？

评分标准：

- 只答"调大堆内存"或"换 G1"
 - 能从 GC 日志中提取分析线索，给出调优决策依据
 - 能综合评估"调参数 vs 改代码内存泄漏 vs 增加机器"的收益
-

题 4：HotSpot JIT 编译的坑

主问题：你了解 JIT 编译中的"逃逸分析"和"栈上分配"吗？你有没有在生产环境遇到过"同一段代码预热前后性能差一个数量级"的情况？你们怎么做的预热？

追问方向：

- `-XX:+PrintCompilation` 输出的日志你会怎么解读？
- C1 和 C2 编译器分别在什么时候介入？什么场景下 C2 编译反而更慢（如大量 deoptimization）？
- 你的微服务在发布后是否需要预热？如果有，你们的预热策略是什么？

评分标准：

- 不知道或不了解
 - 能讲清概念但没实战经验
 - 能结合线上 JIT 编译日志或 deoptimization 案例说明
-

第三部分：数据库与存储（权重 20%）

题 5：千万级用户系统的 SQL 优化

你的用户系统支撑 400+ 游戏接入，服务了大量用户。

主问题：假设你们的用户表（MySQL）已经达到 5000 万行，索引加了但慢查询仍然很多。请从索引设计、SQL 改写、架构层面三个维度，说说你的优化思路。特别注意：你之前用的是 SQL Server + DDD 架构，和 MySQL 在优化器行为上有哪些差异？

追问方向：

- SQL Server 和 MySQL 在查询优化器上的关键差异是什么？（如统计信息更新策略、索引选择逻辑）
- `SELECT * FROM user WHERE id IN (...)` 一千个 ID，MySQL 优化器会怎么执行？会不会走索引？会不会出现 index dive 问题？
- 如果必须分库分表，你会选什么中间件？sharding key 怎么选？

评分标准：

- 只答"加索引"
 - 能分析慢查询日志、合理评估索引选择性、区分覆盖索引和回表
 - 能从优化器行为差异、分库分表方案取舍、以及"分库分表带来的分布式事务/跨 shard 查询"问题出发
-

题 6：Redis 的深度使用

你在多个项目中用 Redis 做缓存层。

主问题：你说用 Redis 做"用户系统"的数据缓存。请描述你们的缓存更新策略（Cache-Aside? Write-Through? Write-Behind? ），以及在高并发下如何避免缓存雪崩、缓存穿透、缓存击穿？有没有遇到过 Redis 大 Key 或热 Key 导致集群倾斜的问题？

追问方向：

- 你用 Redis 集群还是哨兵？选型依据是什么？
- 如果缓存和数据库双写不一致，你们怎么处理？删缓存还是更新缓存？先删还是后删？
- 你接触的 Redis 最大数据量是多少？单实例内存多大？过期键的淘汰策略你怎么配？

评分标准：

- 只答"把热点数据放 Redis"
 - 能讲清缓存模式、穿透/击穿/雪崩的应对策略
 - 能结合实际集群架构、大 Key 拆分经验、数据一致性权衡做出深度分析
-

第四部分：微服务架构实战（权重 20%）

题 7：微服务拆分与分布式事务

你在代理商系统中把一个业务拆成了 5 个微服务（账户、广告位、排期、结算、效果），在同城游中台也做了类似的拆分。

主问题：你提到代理商系统按照业务功能拆了 5 个服务——请复盘一下，这个拆分粒度在实际运行中有什么问题和教训？比如有没有出现"本想解耦，结果变成了分布式单体"的情况？如果让你重做，你会调整吗？

追问方向：

- 拆得太细导致一个业务操作需要跨 5 个服务协调，你怎么处理这种分布式事务？
- 服务拆分后，你的 API 网关（Zuul/Gateway）有什么性能瓶颈？有没有做过聚合层的优化？

- 你们怎么定义"服务拆分的边界"？你用什么方法论的——DDD 限界上下文？业务能力？还是团队拓扑？

评分标准：

- 只答"拆了之后耦合低"
 - 能坦诚说出拆分过细带来的事务/性能问题
 - 能从实际踩坑出发，结合方法论（DDD/康威定律）给出边界定义的经验原则
-

题 8：CI/CD 与 DevOps 实施

你在多个项目中都提到了 GitLab CI/CD 和自动化运维。

主问题：你们的 CI/CD 流水线具体怎么设计的？从代码提交到生产上线，有哪些卡点（代码审查、自动化测试、灰度发布、回滚策略）？你们的测试覆盖率要求是多少？

追问方向：

- 你们的灰度发布是怎么实现的？网关层配置路由权重还是用 K8s Service 的 label selector？有没有和金丝雀发布结合？
- 有没有遇到"CI 过了但上线炸了"的情况？根因是什么，后来怎么加固的？
- 你们的 Docker 镜像是怎么做分层优化的？基础镜像是什么？编译和运行环境怎么分离？

评分标准：

- 只答"用 Jenkins/GitLab CI 自动部署"
 - 能描述完整的发布流程和关键卡点
 - 能从"CI 通过 ≠ 质量保证"出发，说明质量内建的策略和灰度/回滚机制
-

第五部分：技术领导力与管理（权重 15%）

题 9：50 人团队管理与考核

你管理过 50 人全功能团队，实施了 KPI/KPA/OKR 等多种考核方式。

主问题：你提到在团队中实施过 KPI、KPA、OKR 三种考核方式——分别在什么阶段、为什么切换？你认为对于不同角色（后端、前端、客户端、测试），考核方式应该有什么区别？有没有遇到过"考核导致行为走偏"的情况？

追问方向：

- OKR 在研发团队的实施最大的阻力是什么？你怎么让产品/测试也能参与进来？

- 你说"分管产品技术"，产品和研发的矛盾你怎么协调？有没有具体案例？
- 有没有开掉过人？原因是什么？你怎么判断一个人是"能力问题"还是"意愿问题"？

评分标准：

- 只答"因团队而异"或泛泛而谈
 - 能说出每种考核的具体适用场景和切换原因
 - 能从真实案例出发，展现"考核-行为-结果"的闭环思考，并有纠偏经验
-

题 10：技术债与架构演进

主问题：你在畅唐网络对原有 SOA (webservice) 做了向 .NET Core RESTful + Docker 的改造升级。这类架构升级必然涉及业务中断风险和技术债取舍。你是怎么说服业务方和上级接受这类"看不到业务价值"的技术改造的？有没有改造到一半发现行不通、被迫回退的情况？

追问方向：

- 你如何和老旧系统做兼容过渡？有没有用到绞杀者模式 (Strangler Fig) ？
- 你怎么平衡"业务需求压力"和"技术债偿还"的时间分配？团队内部怎么分工的？
- 你判断"这个技术债必须现在还"和"可以再拖一拖"的标准是什么？

评分标准：

- 只答"技术升级理所当然"
 - 能说清推进策略和妥协方案
 - 能从实际案例出发，展现"技术决策×商业权衡×团队执行力"的三维思考
-

第六部分：区块链专项（加分项 / 权重 N/A）

题 11：POA 联盟链的工程实践（Bonus）

你在数字权益平台中基于以太坊搭建了 POA 联盟链。

主问题：你们选 POA 而不是 PBFT/Raft 的共识，是出于什么考虑？POA 的签名者在 4-7 个节点之间——你们怎么处理签名者节点的故障和替换？在联盟链场景下，你们为什么没用 Hyperledger Fabric 而是自建以太坊 POA 链？

追问方向：

- ERC20 和 ERC721 合约中你们有没有自定义扩展逻辑？合约升级是怎么做的（proxy pattern？）？

- Uniswap 合约你是自己部署的还是 Fork 的？有没有改动 AMM 曲线参数？
- 区块链的数据存储层（LevelDB）在大批量事件查询时性能如何？你们怎么优化的？

评分标准：

- 只答技术选型结论
- 能说出 POA vs PBFT 在联盟链场景下的工程差异
- 能从共识故障恢复、合约可升级性、存储查询性能等完整链路说明